



資料結構

Data Structure

Lab11

姓名：王顥程
學號：114310215

Lab11

Exercise: Complement Level Sum function

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

const int EMPTY = NULL;
class TreeNode {
public:
    int value;    // 節點值
    TreeNode* left; // 左子節點
    TreeNode* right; // 右子節點

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

class BinaryTree {
public:
    TreeNode* root; // 樹根節點

    BinaryTree() : root(nullptr) {}

    TreeNode* buildTree(const vector<int>& arr) { // 建立樹
        if (arr.empty() || arr[0] == EMPTY) return nullptr;

        queue<TreeNode*> q;
        root = new TreeNode(arr[0]);
        q.push(&root);

        size_t i = 1;
        while (!q.empty() && i < arr.size()) {
            TreeNode** nodePtr = q.front();
            q.pop();

            // 左子節點
            if (i < arr.size()) {
                if (arr[i] != EMPTY) {
                    (*nodePtr)->left = new TreeNode(arr[i]);
                    q.push(&((*nodePtr)->left));
                }
            }
            i++;
        }
    }
};
```

```

    }
    // 右子節點
    if (i < arr.size()) {
        if (arr[i] != EMPTY) {
            (*nodePtr)->right = new TreeNode(arr[i]);
            q.push(&((*nodePtr)->right));
        }
        i++;
    }
}
return root;
}

void Depth_first_search(TreeNode* node) {
    if (node == nullptr) return;
    cout << node->value << " ";
    Depth_first_search(node->left);
    Depth_first_search(node->right);
}

void Breadth_first_search(TreeNode* root) {
    if (root == nullptr) return;
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        TreeNode* current = q.front();
        q.pop();
        cout << current->value << " ";
        if (current->left) q.push(current->left);
        if (current->right) q.push(current->right);
    }
}

// 計算特定層數總和
void printLevelSum(int targetLevel) {
    // 若樹為空或輸入小於 0 的不合法層數
    if (root == nullptr || targetLevel < 0) {
        cout << "The layer exceeds the tree height." << endl;
        return;
    }

    queue<TreeNode*> q;

```

```

q.push(root);

int currentLevel = 0; // 根節點為第 0 層

while (!q.empty()) {
    int levelSize = q.size(); // 當前層的節點數量
    int currentLevelSum = 0;

    // 遍歷當前層的所有節點
    for (int i = 0; i < levelSize; i++) {
        TreeNode* current = q.front();
        q.pop();

        currentLevelSum += current->value;

        if (current->left) q.push(current->left);
        if (current->right) q.push(current->right);
    }

    // 若目前層數等於目標層數, 輸出結果並結束函數
    if (currentLevel == targetLevel) {
        cout << "The sum of level " << targetLevel << " is: " << currentLevelSum << endl;
        return;
    }

    currentLevel++;
}

// 防禦性檢查 2: 若迴圈跑完還沒 return, 代表 targetLevel 超過樹的高度
cout << "The layer exceeds the tree height." << endl;
}
};

int main() {
    BinaryTree tree;
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, NULL, NULL, 10, 11, NULL, NULL };
    tree.buildTree(arr);

    // 1. 輸出 BFS Result
    cout << "BFS Result: ";
    tree.Breadth_first_search(tree.root);
    cout << endl;
}

```

// 2. 提示使用者輸入層數

```
int inputLevel;  
cout << "Please enter the layer to query, starting from 0: ";  
cin >> inputLevel;
```

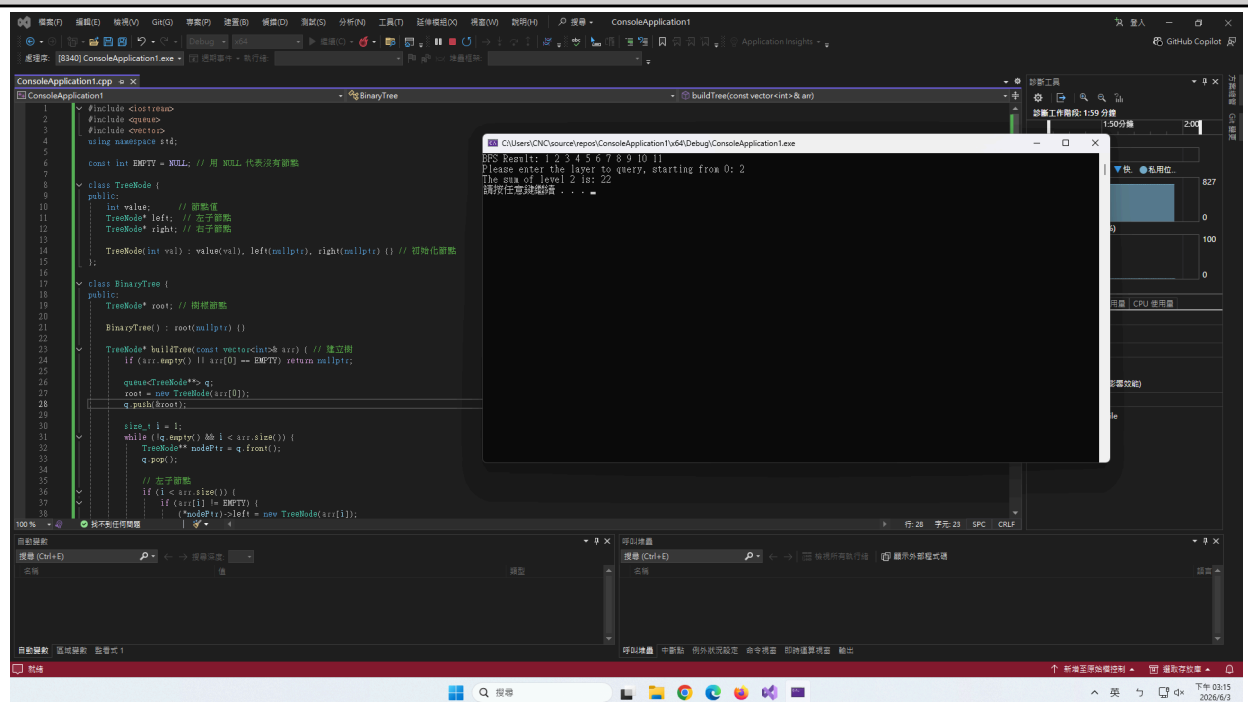
// 3. 計算並輸出結果

```
tree.printLevelSum(inputLevel);
```

```
system("pause");
```

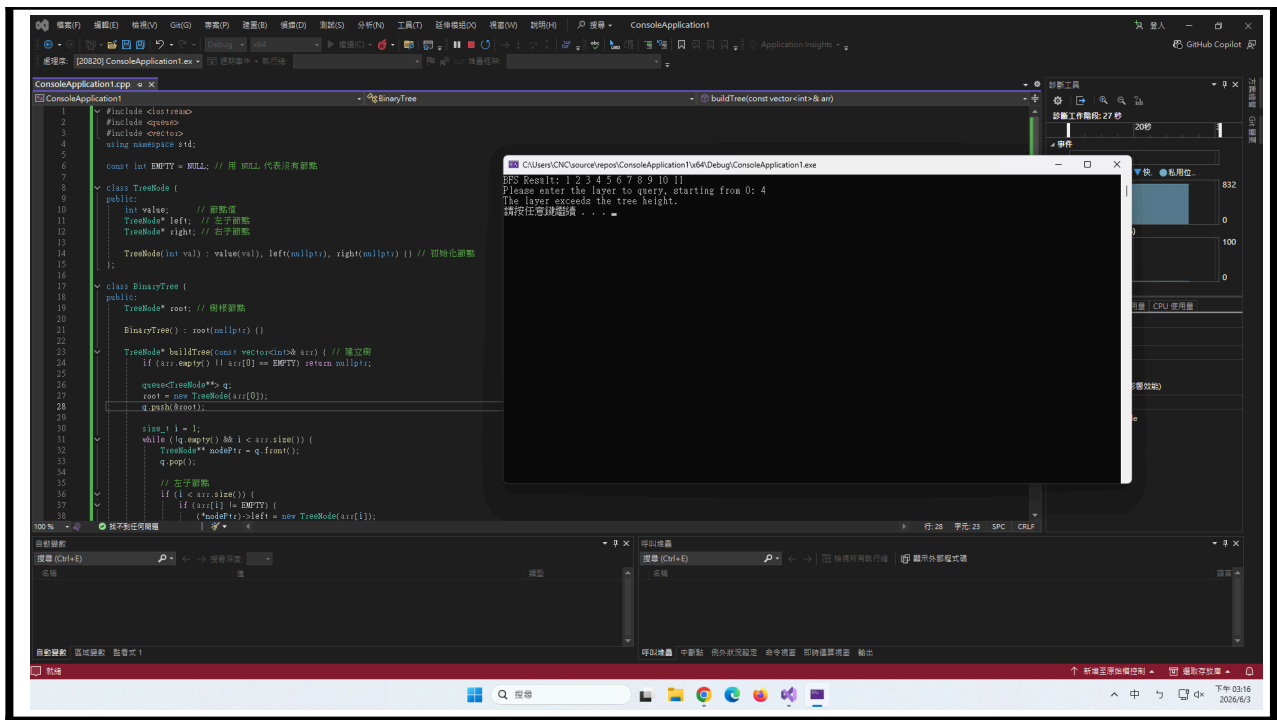
```
return 0;
```

```
}
```



```
C:\Users\CNC\source\repos\ConsoleApplication1\x64\Debug\ConsoleAppli  
BFS Result: 1 2 3 4 5 6 7 8 9 10 11  
Please enter the layer to query, starting from 0: 2  
The sum of level 2 is: 22  
請按任意鍵繼續 . . .
```

```
C:\Users\CNC\source\repos\ConsoleApplication1\x64\Debug\ConsoleAppl  
BFS Result: 1 2 3 4 5 6 7 8 9 10 11  
Please enter the layer to query, starting from 0: 4  
The layer exceeds the tree height.  
請按任意鍵繼續 . . .
```



Lab01-Q1. (題目編號)

Code

Result

Discussion

Lab01-Q#. (題目編號)
Briefly describe the problem. (題目簡述)
Code

Paste your code here. Please use Pygments to highlight your code. (請使用Pygments轉換程式碼格式後再貼上)		Paste your code here. Please use Pygments to highlight your code. (請使用Pygments轉換程式碼格式後再貼上)	
result			
(a)	(b)	(c)	
	Show your result in tables or screenshots. (請使用表格或是截圖呈現結果。)		
Discussion			
Discuss and conclude your results, or answer questions. (實驗討論、結論或回答問題。)			